**MATLAB Simulation of Superheterodyne Receiver for AM Communication System with Frequency Mixing and Filtering.**

**Aim:** To simulate a superheterodyne receiver for an AM communication system using MATLAB. To study frequency mixing, intermediate-frequency (IF) filtering, and recovery of the original message signal.

**Software Required:**MATLAB

**Program:**

```
clc;
clear;
close all;
% MATLAB Simulation of Superheterodyne Receiver
% for AM Communication System with Frequency Mixing and Filtering
%% Step 1: Define system parameters
Fs = 200000;         % Sampling frequency (Hz)
T = 0.02;            % Simulation duration (seconds)
t = 0:1/Fs:T-1/Fs;    % Time vector
N = length(t);
Ac = 10;             % Carrier amplitude
Am = 5;              % Message amplitude
fc = 20000;          % Received carrier frequency (Hz)
fm = 1000;           % Message frequency (Hz)
m = Am/Ac;           % Modulation index
fIF = 5000;          % Intermediate frequency (Hz)
fLO = fc - fIF;      % Local oscillator frequency (Hz)
```

```
%% Step 2: Generate message and AM signal
message = Am*cos(2*pi*fm*t);
received_AM = Ac*(1 + m*cos(2*pi*fm*t)) ...
            .* cos(2*pi*fc*t);

%% Step 3: Generate local oscillator signal
local_oscillator = cos(2*pi*fLO*t);
%% Step 4: Frequency mixing
mixer_output = received_AM .* local_oscillator;
%% Step 5: Intermediate-frequency (IF) filtering
% Use an FFT-based ideal bandpass filter.
% Retain frequencies around +fIF and -fIF.
f = (-N/2:N/2-1)*(Fs/N);
MIX_FFT = fftshift(fft(mixer_output));
IF_mask = (abs(f) >= fIF-1500) & ...
        (abs(f) <= fIF+1500);
IF_FFT = MIX_FFT .* IF_mask;
IF_signal = real(ifft(ifftshift(IF_FFT)));
%% Step 6: Coherent demodulation at the IF stage
IF_oscillator = cos(2*pi*fIF*t);
detector_output = IF_signal .* IF_oscillator;
%% Step 7: Low-pass filtering
% Retain the recovered baseband message frequencies.
```

```
DET_FFT = fftshift(fft(detector_output));
LPF_mask = abs(f) <= 2000;
LPF_FFT = DET_FFT .* LPF_mask;
baseband_signal = real(ifft(ifftshift(LPF_FFT)));
%% Step 8: Recover the original message
% Remove the DC component and compensate for mixing gain.
recovered_message = (baseband_signal - mean(baseband_signal)) ...
                * 4/(Ac*m);
%% Step 9: Display calculated frequencies
fprintf('SUPERHETERODYNE AM RECEIVER SIMULATION\n');
fprintf('--------------------------------------\n');
fprintf('Carrier frequency       = %.2f kHz\n', fc/1000);
fprintf('Message frequency       = %.2f kHz\n', fm/1000);
fprintf('Local oscillator freq.  = %.2f kHz\n', fLO/1000);
fprintf('Intermediate frequency  = %.2f kHz\n', fIF/1000);
fprintf('Modulation index        = %.2f\n', m);
fprintf('Simulation completed successfully.\n');
%% Step 10: Plot time-domain waveforms
figure('Name','Superheterodyne Receiver Time Signals', ...
     'NumberTitle','off');
nPlot = round(0.005*Fs);
tp = t(1:nPlot);
subplot(4,1,1);
```

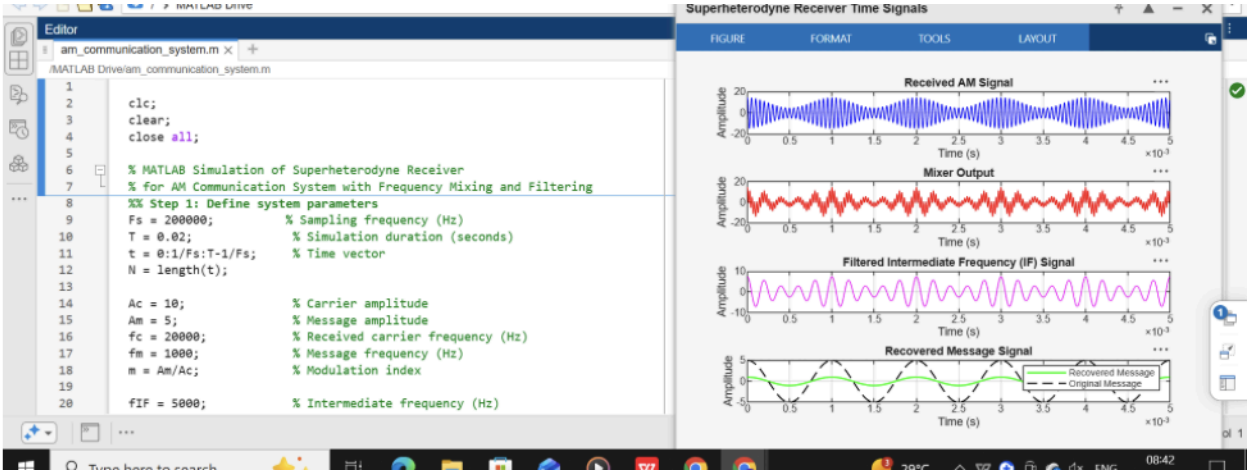
```
plot(tp, received_AM(1:nPlot), 'b');
grid on;
title('Received AM Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(4,1,2);
plot(tp, mixer_output(1:nPlot), 'r');
grid on;
title('Mixer Output');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(4,1,3);
plot(tp, IF_signal(1:nPlot), 'm');
grid on;
title('Filtered Intermediate Frequency (IF) Signal');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(4,1,4);
plot(tp, recovered_message(1:nPlot), 'g', ...
    'LineWidth', 1.2);
hold on;
plot(tp, message(1:nPlot), 'k--', 'LineWidth', 1.0);
```

```
hold off;
grid on;
title('Recovered Message Signal');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Recovered Message','Original Message');
%% Step 11: Plot frequency spectrum
figure('Name','Frequency Mixing and Filtering Spectrum', ...
    'NumberTitle','off');
% Positive-frequency spectrum of received AM signal
RX_FFT = fft(received_AM);
f_pos = (0:floor(N/2))*Fs/N;
RX_mag = abs(RX_FFT(1:floor(N/2)+1))/N;
RX_mag(2:end-1) = 2*RX_mag(2:end-1);
subplot(3,1,1);
plot(f_pos/1000, RX_mag, 'b');
grid on;
title('Received AM Frequency Spectrum');
xlabel('Frequency (kHz)');
ylabel('Magnitude');
xlim([0 40]);
% Positive-frequency spectrum of mixer output
MIX_mag = abs(fft(mixer_output));
```

```
MIX_mag = MIX_mag(1:floor(N/2)+1)/N;
MIX_mag(2:end-1) = 2*MIX_mag(2:end-1);
subplot(3,1,2);
plot(f_pos/1000, MIX_mag, 'r');
grid on;
title('Mixer Output Frequency Spectrum');
xlabel('Frequency (kHz)');
ylabel('Magnitude');
xlim([0 40]);
% Positive-frequency spectrum of filtered IF signal
IF_MAG = abs(fft(IF_signal));
IF_MAG = IF_MAG(1:floor(N/2)+1)/N;
IF_MAG(2:end-1) = 2*IF_MAG(2:end-1);
subplot(3,1,3);
plot(f_pos/1000, IF_MAG, 'm');
grid on;
title('Filtered IF Frequency Spectrum');
xlabel('Frequency (kHz)');
ylabel('Magnitude');
xlim([0 15]);
```

**output:**




**Result:** The superheterodyne AM receiver was successfully simulated using MATLAB. Frequency conversion, IF filtering, and recovery of the original message signal were demonstrated.